Capstone Project - AJIO E-Commerce Automation Testing

Prepared by: Arun N

Project Information

Field	Details
Project Type	Enterprise E-Commerce Automation Testing
Application	AJIO (https://www.ajio.com)
Automation Tool	Playwright
Programming Language	JavaScript
Framework Design	Page Object Model (POM)
Total Planned Test Cases	120+

Project Overview

This capstone project focuses on building a **scalable automation testing framework** for the AJIO e-commerce platform using **Playwright with JavaScript**. Designed with the **Page Object Model (POM)**, the framework ensures long-term **readability, reusability, and maintainability** of test scripts.

The testing plan validates **critical customer workflows** such as authentication, homepage navigation, product search, filtering, catalog and detail checks, cart handling, checkout, address and shipping, user profile management, and customer support. It also incorporates **responsive UI validation, API monitoring, regression testing, and cross-browser execution** to guarantee stability across updates.

By emphasizing **enterprise-grade practices** including reusable architecture, robust automation, and reporting with Playwright HTML and Allure, supported by CI/CD integration and debugging evidence, the project demonstrates a professional QA framework suitable for both capstone requirements and real-world application quality assurance.

Why AJIO Was Selected

AJIO was chosen as the application under test because it is a **real-world e-commerce platform** with dynamic UI components, modern frontend architecture, responsive layouts, and realistic shopping workflows — making it ideal for **enterprise-level automation testing**.

The platform provides multiple automation opportunities such as **catalog validation, search and filtering workflows, product detail checks, cart and bag management, promotions and pricing validation, responsive UI testing, cross-browser execution, and API/network monitoring**.

AJIO also simulates real-world automation challenges including **dynamic locators, asynchronous page loading, session handling, dynamic product listings, responsive layout behavior, and multi-browser compatibility**.

To ensure safe and repeatable automation, the project deliberately avoids sensitive areas such as **OTP dependency, real payment transactions, backend modifications, and the use of real customer information**.

Project Objectives

The objective of this project is to design and implement a **maintainable enterprise-level automation testing framework** for the AJIO e-commerce platform using **Playwright with JavaScript**. Built on the **Page Object Model (POM)**, the framework ensures scalability, reusability, and long-term stability of test scripts.

The testing plan validates **critical business workflows** including homepage navigation, product discovery, catalog and detail checks, shopping bag and cart functionality, promotions and pricing consistency, wishlist behavior, responsive UI validation, API monitoring, and cross-browser execution.

In addition, the project emphasizes **reusable architecture, stable automation execution, enterprise QA practices, reporting and debugging support, data-driven testing, and scalable POM implementation**, ensuring both academic capstone requirements and professional QA standards are met.

Technology Stack

Category	Tools / Technologies
Automation Framework	Playwright
Language	JavaScript
Runtime Environment	Node.js
Design Pattern	Page Object Model (POM)
IDE	Visual Studio Code
Reporting	HTML Report & Allure Report
Version Control	Git & GitHub
CI/CD	GitHub Actions

Scope of Testing

Category	Description
Safe Boundary	Homepage, navigation, product catalogue, filters, sorting, product detail validation, cart functionality, promotions, responsive UI checks, API monitoring, and cross-browser validation.
Out of Scope	Real payment confirmation, real order placement, backend database validation, admin activities, and use of real customer personal information.
Safe Boundary	Tests validate workflows up to safe checkpoints such as product selection, cart updates, visible pricing, and form validation without triggering real transactions.

Services & Test Case Breakdown (8 × 15 = 120+ Test Cases)

No.	Module / Service	Coverage Focus	Count	Severity
1	Homepage & Navigation Validation	Header, menus, banners, footer links, navigation flow, and redirection validation.	15	High
2	User Access & Session Validation	Login entry behaviour, guest restrictions, session visibility, and validation messages.	15	High
3	Product Discovery & Catalogue Experience	Product search, category browsing, filters, sorting, pagination, and catalogue validation.	15	Critical
4	Product Listing & Product Detail Validation	Product title, images, price, offers, size options, and recommendation validation.	15	High
5	Shopping Bag & Cart Management	Add/remove items, quantity updates, subtotal validation, and persistence checks.	15	Critical
6	Wishlist & Saved Items	Wishlist entry points, login restrictions, and saved item behaviour.	15	Medium
7	Checkout & Shipping Address Validation	Address validation, checkout navigation, delivery information, and validation handling.	15	Critical
8	API, Responsive & Cross-Browser Validation	API/network validation, responsive layouts, viewport validation, and multi-browser execution.	15	High

The planned 120+ test cases ensure comprehensive coverage across critical business workflows and validation scenarios.

Detailed Module Coverage

M1 – Homepage & Navigation Validation

Validate homepage loading, banners, menus, category navigation, footer links, redirects, and overall navigation flow to ensure users can access and browse the platform seamlessly.

M2 – User Access & Session Validation

Validate login entry behavior, guest restrictions, session visibility, validation messages, and basic session persistence across navigation and refresh actions.

M3 – Product Discovery & Catalogue Experience

Validate product search, category browsing, filters, sorting, pagination, product cards, and result consistency to ensure smooth product discovery workflows.

M4 – Product Listing & Product Detail Validation

Validate product titles, images, prices, offers, size options, availability labels, recommendations, and consistency between listing and product detail pages.

M5 – Shopping Bag & Cart Management

Validate add-to-cart functionality, remove item actions, quantity updates, subtotal calculations, bag count validation, and cart persistence after refresh or navigation.

M6 – Wishlist & Saved Items

Validate wishlist entry points, saved item behavior, guest access restrictions, login redirection, and wishlist icon state validation.

M7 – Checkout & Shipping Address Validation

Validate checkout navigation, address input validation, shipping details, delivery information, and form validation workflows up to safe transaction boundaries.

M8 – API, Responsive & Cross-Browser Validation

Validate API/network responses, request status monitoring, responsive layouts across different viewports, and consistent behavior across Chromium, Firefox, and WebKit browsers.

Planned Testing Types

Testing Type	Purpose
Functional Testing	Validate end-to-end business workflows
Validation Testing	Validate invalid inputs and restrictions
UI Assertion Testing	Verify visible UI elements, labels, and content
Regression Testing	Ensure existing workflows remain stable after updates
Session Testing	Validate cart/session persistence after refresh or navigation
API & Network Testing	Observe network calls, response status, and request validation
Responsive Testing	Validate usability across desktop, tablet, and mobile viewports
Cross-Browser Testing	Execute workflows across Chromium, Firefox, and WebKit

---

Framework Design & Approach

The automation framework is implemented using **Playwright with JavaScript**, following the **Page Object Model (POM)** for scalability and maintainability.

Key features include:

- Reusable page classes and modular test structure
- Utility/helper functions and data-driven testing
- Screenshot and trace generation for debugging

- Cross-browser execution and API/network monitoring
- Comprehensive reporting with Playwright HTML and Allure

The framework relies on **stable Playwright locators** such as getByRole(), getByText(), getByPlaceholder(), and locator(), while avoiding fragile selectors and deeply nested XPath expressions to ensure robust and reliable automation.

Planned Project Structure

Folder / File	Purpose
pages/	Stores reusable Page Object Model (POM) classes
tests/	Contains Playwright test specification files
utils/	Reusable helper functions, waits, and utilities
test-data/	Stores reusable test data and JSON datasets
screenshots/	Stores execution screenshots and failure evidence
playwright-report/	Stores Playwright HTML execution reports
allure-results/	Stores Allure reporting results
docs/	Contains project proposal and project documentation
README.md	Project overview, setup steps, and execution guide
package.json	Manages project dependencies and scripts
playwright.config.js	Central Playwright configuration file
.gitignore	Excludes unnecessary files from Git tracking

Test Data Strategy

Since AJIO is a live public platform, the framework uses **safe, replaceable test data** to avoid real transactions or sensitive customer information.

Key practices include:

- Using **dummy names, emails, contacts, and postal codes** only for safe form validation.
- Treating **visible product names and categories** as configurable test data rather than hard-coding them.
- Employing **search keywords** (e.g., shoes, kurta, jeans, bag, watch) based on current site availability.
- Avoiding reliance on a single product that may be removed or become unavailable.
- Maintaining **separate data files** so keywords, sizes, and inputs can be updated without editing every test.

This strategy ensures **flexibility, safety, and maintainability** while supporting reliable automation across evolving site content.

Reporting & Evidence Strategy

The framework generates **professional execution evidence** through:

- Playwright HTML and Allure reports
- Screenshots for failed scenarios
- Trace files for debugging
- GitHub Actions execution logs
- Cross-browser execution reports

These reports provide clear visibility into **pass/fail statistics, execution trends, failure details, flaky test detection, screenshot evidence, and debugging information**, ensuring both transparency and reliability in test outcomes.

Risks & Mitigation

Risk	Mitigation
Dynamic UI changes	Use stable Playwright locators and maintain selectors inside page objects
Product availability changes	Use configurable product keywords instead of hardcoded products
Login restrictions or OTP	Avoid unsafe OTP dependency and validate guest restrictions instead
Slow public website response	Use Playwright auto-waiting and proper timeout handling
Promotion text changes	Validate general offer visibility instead of exact promotional text

---

Expected Deliverables

The project will deliver a **professional-grade automation framework** showcasing enterprise QA practices. Key outputs include:

- **Playwright automation framework** with organized **Page Object Model (POM)** architecture
- Automated, **reusable test suites** covering critical workflows
- **API and network validation** implementation
- **Cross-browser execution support**
- **Playwright HTML and Allure reports** for detailed insights
- **Screenshots and trace evidence** for debugging
- A **GitHub repository** with documentation and CI/CD integration
- **Service-wise automation coverage** demonstrating breadth and reliability

Conclusion

This capstone project demonstrates the design and implementation of a **scalable, maintainable automation testing framework** for a real-world e-commerce platform using **Playwright with JavaScript**.

The framework validates **critical business workflows** including UI components, product discovery, cart functionality, responsive behavior, and end-to-end shopping scenarios, while adhering to **industry-standard QA practices**.

It highlights practical **SDET skills** such as framework design, reusable scripting, reporting, debugging, API validation, regression support, and cross-browser testing, showcasing the application of **modern automation technologies** in an enterprise context.